
132. Repair Service Management System

Problem Statement

An appliance repair company wants customers to raise repair requests with an appliance photo, which are then assigned to a technician. A technician should update only the status of requests assigned to them, and a customer should be able to view only their own requests.

Objectives

- Design Customer, Technician and RepairRequest schemas.
- Implement image upload for the appliance using Multer.
- Implement ownership-based and role-based authorization.
- Implement status-workflow updates for technicians.
- Validate repair request data before saving.

Outcomes

- Customers can raise repair requests with a photo of the appliance.
- Technicians can update only requests assigned to them.
- Customers can track the status of their own requests only.
- Uploaded photos are stored and linked correctly in MongoDB.

Deliverables

- Express.js REST API for repair requests.
- Multer configuration for appliance photo upload.
- MongoDB/Mongoose database with referenced collections.
- Postman/Thunder Client collection.
- Environment configuration for DB and upload path.

Suggested Frontend (Basic)

- React frontend (request form, status tracker, technician panel).
- Core pages: Login/Register, main List/Dashboard view, Add/Update form, and a Details page.
- Frontend only calls the backend REST APIs (Axios/Fetch) for data; all validation and business logic stay in Express.js and MongoDB.

Suggested Backend Requirements

- RepairRequest schema referencing Customer and Technician with photoPath and status.
- POST /requests (Multer image upload), PATCH /requests/:id/status (assigned technician only).
- GET /requests/my (customer), GET /requests/assigned (technician).
- Ownership-based and role-based authorization middleware.
- JWT authentication middleware.
- Validation for required request fields.
- MongoDB Atlas connection and .env configuration.

Deployment Note

Deploy the backend on Render/Railway with MongoDB Atlas, PORT and JWT secret configured via environment variables; deploy the React frontend on Vercel/Netlify and connect it to the live backend using an environment-based API base URL.